

HelixCode — Open Issues Tracker

Per Constitution §11.4.15 (Item-status tracking) + §11.4.16 (Item-type tracking) + §11.4.19 (Fixed-document column-alignment) + CONST-057 (Type-aware closure vocabulary) + CONST-058 (Reopened-source attribution).

Authoritative resumption ledger: docs/CONTINUATION.md (CONST-044). This file complements it with item-level granularity for currently-open work.

Status vocabulary (closed set): Queued | In progress | Ready for testing | In testing | Reopened | Fixed/Implemented/Completed (→ Fixed.md)

Type vocabulary (closed set): Bug | Feature | Task

Prefix convention

Round 189 (2026-05-19) introduced per-project / per-submodule ID prefixes replacing the legacy ISSUE-NNN flat namespace. New items **MUST** use the prefix matching their scope; cross-cutting items affecting two or more submodules (or any meta-repo concern) live under HXC. Numeric portion is per-prefix (each prefix starts at 001). Legacy ISSUE-NNN IDs renamed forward-only per CONST-043; historical close-out narratives in docs/CONTINUATION.md preserve original IDs verbatim, and docs/Fixed.md annotates each closure with (ex-ISSUE-NNN) for git-history traceability.

Prefix	Scope	Source
HXC	HelixCode root project (project-wide, multi-submodule, governance, infrastructure)	this repo
HXA	HelixAgent submodule	dependencies/ HelixDevelopment/ HelixAgent (when

Prefix	Scope	Source
		present) / helix_agent/ tree
HXM	HelixMemory submodule	dependencies/ HelixDevelopment/ HelixMemory
HXL	HelixLLM submodule	dependencies/ HelixDevelopment/ HelixLLM
HXQ	HelixQA submodule	dependencies/ HelixDevelopment/ HelixQA (helix_qa/)
HXS	HelixSpecifier submodule	dependencies/ HelixDevelopment/ HelixSpecifier
HXO	HelixOrchestrator (= LLMOrchestrator) submodule	dependencies/ HelixDevelopment/ LLMOrchestrator
HXV	HelixVerifier (= LLMsVerifier) submodule	dependencies/ HelixDevelopment/ LLMsVerifier
HXD	HelixDocProcessor (= DocProcessor) submodule	dependencies/ HelixDevelopment/ DocProcessor
HXI	HelixI18n (when added)	tba
PLN	Planning submodule (vasic- digital)	dependencies/vasic- digital/Planning
VEN	VisionEngine submodule (HelixDevelopment)	dependencies/ HelixDevelopment/ VisionEngine
SLF	SelfImprove submodule (vasic- digital)	dependencies/vasic- digital/SelfImprove
STO	Storage submodule (vasic-digital)	dependencies/vasic- digital/Storage
OPS	LLMOps submodule (vasic- digital)	dependencies/vasic- digital/LLMOps
VDB	VectorDB submodule (vasic- digital)	dependencies/vasic- digital/VectorDB
OBS		

Prefix	Scope	Source
	Observability submodule (vasic-digital)	dependencies/vasic-digital/Observability
MCP	MCP_Module submodule (vasic-digital)	dependencies/vasic-digital/MCP_Module
MSG	Messaging submodule (vasic-digital)	dependencies/vasic-digital/Messaging
MDW	Middleware submodule (vasic-digital)	dependencies/vasic-digital/Middleware
PLG	Plugins submodule (vasic-digital)	dependencies/vasic-digital/Plugins
STR	Streaming submodule (vasic-digital)	dependencies/vasic-digital/Streaming
WAT	Watcher submodule (vasic-digital)	dependencies/vasic-digital/Watcher
CNV	conversation submodule (vasic-digital)	dependencies/vasic-digital/conversation
AUT	Auth submodule (vasic-digital)	dependencies/vasic-digital/Auth
LZY	Lazy submodule (vasic-digital)	dependencies/vasic-digital/Lazy
ATP	AutoTemp submodule (vasic-digital)	dependencies/vasic-digital/auto_temp
PLI	PliniusCommon submodule (vasic-digital)	dependencies/vasic-digital/plinius_common
CHL	challenges submodule (vasic-digital)	challenges/
CNT	containers submodule	containers/
SEC	security submodule	security/
PAN	panoptic submodule	panoptic/

For submodules not listed above, default to the first 3 letters of the submodule name, uppercase (e.g. Watcher → WAT). Document the new prefix in this table on first use.

Legacy → new mapping (round 189)

Old ID	New ID	Scope rationale
ISSUE-001	VEN-001	VisionEngine helix-gitlab URL
ISSUE-002	VEN-002	VisionEngine vasic-digital-github fork divergent
ISSUE-003	HXL-001	HelixLLM analysis_test.go hardcoded path
ISSUE-004	HXL-002	HelixLLM TOON WriteT00N 500
ISSUE-005	HXC-001	CONST-052 rename programme (project-wide)
ISSUE-006	HXC-002	Round-74 residual LOGIC FAILs (multi-submodule cross-cutting)
ISSUE-007	HXC-003	CONST-046 migration backlog (project-wide)
ISSUE-008	HXQ-001	helix_qa TestPerformance flake
ISSUE-009	HXA-001	helix_agent handler tests (4)
ISSUE-010	HXA-002	helix_agent debate API drift
ISSUE-011	HXA-003	venice CONST-037 (in helix_agent)

VEN-001 (ex-ISSUE-001) — VisionEngine helix-gitlab remote repo missing (404) — CLOSED (→ Fixed.md)

Status: Completed (→ Fixed.md) **Type:** Task **Discovered:** 2026-05-19 (round 98 — Planning + VisionEngine i18n migration) **Discovered-By:** AI subagent during 4-remote push attempt **Closed-By:** Round 188 (subagent repo-inventory sweep) **Root cause:** The helix-gitlab remote URL in dependencies/HelixDevelopment/VisionEngine/.git/config pointed at git@gitlab.com:HelixDevelopment/visionengine.git — a non-existent group path. The actual GitLab group is helixdevelopment1 (path) / HelixDevelopment (display name). The repository helixdevelopment1/VisionEngine (id 80411994) already existed since 2026-03-19. NOT a missing-repo issue — a URL-misconfiguration issue. **Fix:** git remote set-url helix-gitlab git@gitlab.com:helixdevelopment1/VisionEngine.git in the VisionEngine submodule, then git push helix-gitlab master (FF-safe: local was 46 commits ahead, remote 0 ahead). Push landed at SHA 2d0c35b (verified via git ls-remote helix-gitlab master). The Upstreams recipe push-helix-gitlab.sh references the remote by name (not URL), so it continues to work unchanged. **Evidence:** - glab api projects/helixdevelopment1%2Fvisionengine → id 80411994 OK -

git ls-remote helix-gitlab HEAD →
2d0c35bebb199a9a199fbf899eaeb292e38eaf17 (matches local HEAD) -
Original broken URL still 404s when probed directly (proves URL
was the issue, not perms)

HXL-001 (ex-ISSUE-003) — HelixLLM internal/agents/tools/analysis_test.go hardcoded absolute path

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-19
(round 95 — HelixLLM migration; surfaced as pre-existing failure)
Discovered-By: AI subagent during HelixLLM standalone test
run **Closed-By:** Round 105 (commit a5e56d4 in HelixLLM; meta
pointer fedd152) **Attribution correction:** Originally documented
as helix_agent; actual location is HelixLLM submodule
(dependencies/HelixDevelopment/HelixLLM/internal/agents/tools/).
Commit SHAs 0a84310 resolved there. **Resolution:** Replaced
hardcoded path with t.TempDir() + 2 synthesised fixture files.
Bonus: same bug-pattern discovered in git_test.go (constant
helixLLMRoot + 7 tests) — refactored gitSandbox() signature. 6 tests
now PASS on any host. Mutation verified.

HXL-002 (ex-ISSUE-004) — HelixLLM internal/gateway/middleware TOON WriteT00N returns 500

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-19
(round 95) **Discovered-By:** AI subagent **Closed-By:** Round 105
(commit a5e56d4) **Attribution correction:** Originally documented
as helix_agent; actual location is HelixLLM submodule. Commit
6f11c56 resolved there. **Resolution:** Root cause was vasic-digital/
TOON's round-27 anti-bluff change (Marshal returns
ErrT00NEncodingNotImplemented unconditionally) combined with
WriteT00N treating ANY Marshal error as 500. Fix: fall back to
json.Marshal while preserving application/toon Content-Type
(matches ContentNegotiation middleware). 500 still returned for
genuinely unmarshallable values (channels). 19 middleware tests
now PASS. Mutation verified.

HXC-001 (ex-ISSUE-005) — CONST-052 rename programme: meta-repo directories still PascalCase — CLOSED (→ Fixed.md)

Status: Completed (→ Fixed.md) — see docs/Fixed.md for the full closure record. **Type:** Task **Closure (2026-05-28):** all owned-org submodule LEAF dirs renamed to lowercase snake_case (Phases 1-4: 1-A..1-D Upstreams; 2-A..2-D / 3 / 4 leaf dirs) + all 57 Upstreams/→upstreams/ dirs. Phase 5 (org-grouping dirs dependencies/vasic-digital/ + dependencies/HelixDevelopment/) resolved as a NO-OP per operator decision 2026-05-28 (AskUserQuestion): kept as GitHub-org namespace carve-outs. Section retained as a migration tombstone per §11.4.19; round-343 detail below preserved for history. **Discovered:** 2026-05-15 (CONST-052 cascade landed) **Discovered-By:** Constitution **Evidence:** Meta-repo top-level dirs already snake_case (round-88 sweep). Remaining non-compliance is two layers deeper: dependencies/HelixDevelopment/* + dependencies/vasic-digital/* owned-org submodule dirs (PascalCase), and 59 Upstreams/ dirs inside submodule trees. **Resolution path:** Phased migration per CONST-052 §11.4.29. Round 113 produced the phased plan (f666410, docs/superpowers/specs/2026-05-19-const052-rename-programme-plan.md). Round 343 executed the safe (zero-submodule-go.mod-entanglement) batches.

Round-343 12 chosen snake_case names (operator “agent defaults”): D-1 sequential phases; D-2 helix_development (parent dir, deferred — touches every consumer go.mod); D-3 vasic-digital kept (GitHub-org handle, proper-noun carve-out); D-4 n/a (helix_code already snake_case); D-5 LLMsVerifier/Assets+Website deferred (deployment-wire audit); D-6 mcp_module; D-7 i_llm; D-8 toon; D-9 rag; D-10 cluster-C upstreams strict; D-11 yes co-authored; D-12 one approval per batch.

Round-343 per-batch status:

Batch	Renamed	Status	Evidence
1	HelixDevelopment/ Models → models	LANDED alea3c8	submodule resolves; go build ./ internal/... ./ cmd/... exit 0
2	HelixDevelopment/ DebateOrchestrator → debate_orchestrator	LANDED 416fe8e	go list -m digital.vasic.debate → new path; build exit 0
3	11 vasic-digital/* zero-go.mod-		

Batch	Renamed	Status	Evidence
	consumer leaves (auto_temp, claritas, doc_processor, gandalf_solutions, hyper_tune, i_llm, leak_hub, ouroborous, plinius_common, veritas, vision_engine)	LANDED e813b5c	11 submodule statuses resolve; build exit 0
Deferred	~37 owned-org leaves consumed by helix_agent/ helix_qa/HelixLLM go.mod	DEFERRED	renaming requires submodule-internal go.mod commits entangled with pre- existing uncommitted work — needs dedicated per-submodule rounds
Deferred	parent dirs HelixDevelopment/ →helix_development/ (D-2), vasic- digital/ kept (D-3)	DEFERRED	parent rename touches every consumer go.mod atomically
Deferred	59 Upstreams/ →upstreams/ (cluster C, D-10)	DEFERRED	live inside submodule trees — separate-repo commits

13 of ~58 owned-org leaf renames done this round (zero build breakage). HXC-001 stays In progress pending the deferred submodule-entangled and parent-dir batches.

HXC-002 (ex-ISSUE-006) — Round-74 residual LOGIC-class FAILs (CLOSED)

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-19 (round 74 — release-gate-test.sh creation; classified by round 89) **Discovered-By:** AI release-gate sweep **Closure progress:** - ✓ HelixMemory: closed round 106 (commit 69016df — single-line go.mod fix; 6 FAIL → 0 FAIL) - ✓ vasic-digital/Planning: round 107 NO-OP — 275 PASS / 0 FAIL / 20 SKIP-OK; likely incidentally fixed by round 98 i18n migration - ✓ helix_agent inner: closed round 109 (commit 0f492e98 — 5 test-side bluff fixes, zero production changes) **Evidence:** Round 74 surfaced 26 FAILs across submodules; rounds 82-87 closed 19; this Issue tracked the

residual 7 across 3 submodules. All 3 components closed by rounds 106 + 107 + 109. **Follow-ups surfaced (NEW issues filed):** 4 helix_agent handler tests previously masked by mid-run panic (now visible) + 3 build-failed packages depending on sibling submodule API drift (digital.vasic.debate) + 2 LOGIC FAILs reclassified as cross-cutting work (venice CONST-037 model-wiring + compliance CONST-051 architectural reconciliation). See HXA-001 through HXA-003 (filed below).

HXA-001 (ex-ISSUE-009) — helix_agent handler tests surfaced after round-109 fix

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-19 (round 109) **Discovered-By:** AI subagent (helix_agent LOGIC audit) **Evidence:** Mid-run panic in TestIsProviderAvailable_NotAvailable aborted test binary; round 109's fix unblocked execution, surfacing 4 pre-existing FAILs: TestFormattersHandler_FormatCode_UnsupportedLanguage, TestEmbeddingHandler_WithRealManager, TestGetTaskResources, TestGetTaskLogs. Out of round-109's 5-fix cap. **Resolution path:** Per-handler investigation, similar to round 109's test-side bluff pattern. **Closure:** Fixed round 116 (commit da782d4) — all 4 handler tests fixed; closure recorded in docs/Fixed.md. This ****Status:**** line was stale (Queued) and is corrected here to match Fixed.md + Issues_Summary.md (§11.4.12 sync).

HXA-002 (ex-ISSUE-010) — helix_agent debate/llmprovider sibling-submodule API drift

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-19 (round 109) **Discovered-By:** AI subagent **Investigated:** 2026-05-20 (round 324) — split into a mechanical part and a design-decision part. **Closed:** 2026-05-20 (round 342) **Closure-Ref:** helix_agent commit (round-342 HXA-002 debate API drift) + meta-repo .gitmodules pointer-bump **Investigation finding (operator's explicit ask — moved vs deleted):** The learning/knowledge/recommendations capability tier was **genuinely DELETED, not moved.** git log on the digital.vasic.debate submodule (dependencies/HelixDevelopment/debate_orchestrator, renamed from DebateOrchestrator per CONST-052) shows the orchestrator was rebuilt from scratch — commit 196d0ea "feat: initial DebateOrchestrator reconstruction (Phase 1)". orchestrator/api.go has carried only the slim CreateDebate/

GetStatistics surface since that very first commit (git log --follow orchestrator/api.go = single entry). A tree-wide grep of dependencies/ for KnowledgeRepository, GetRecommendations, ConvertAPIRequest, GetDebateStatus, DefaultMinConsensus, MaxAgentsPerDebate, EnableAgentDiversity found **zero** surviving copies in any digital.vasic.* package or in HelixSpecifier/ HelixMemory. The slim API is the first and only version — the richer tier was a pre-reconstruction artifact that no longer exists anywhere. Per the operator's chosen direction for the deleted case, the helix_agent tests were rewritten down to the slim API. **Resolution:** See docs/Fixed.md row for the full closure narrative (Part-1 import swap + Part-2 slim-API rewrite + score-scale + go.mod rename-drift fix + captured evidence).

HXA-003 (ex-ISSUE-011) — venice TestGetCapabilities model-list drift (CONST-037)

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-19 (round 109) **Discovered-By:** AI subagent **Closed:** 2026-05-19 (round 190) **Closure-Ref:** helix_agent commit (round-190 venice CONST-037 model-list drift) + meta-repo pointer-bump **Evidence:** Test hardcoded venice-uncensored; Venice API returned 75 models with the family rotated to venice-uncensored-1-2 / venice-uncensored-role-play. Per CONST-037 (LLMsVerifier is the single source of truth for model metadata) the assertion violated the no-hardcoded-list rule. **Resolution:** helix_agent/internal/llm/providers/venice/venice_test.go::TestGetCapabilities — replaced assert.Contains(..., "venice-uncensored") and assert.Contains(..., "llama-3.3-70b") with structural assertion: NotEmpty(SupportedModels) plus a substring scan for the venice-uncensored* family. SKIP-OK marker per CONST-035 fires if the entire family disappears (avoids false-positive PASS). Mutation-verified (revert → FAIL with the original drift, restore → PASS).

HXC-004 — Recovery-batch under- verification (40% FAIL rate per round-193 audit)

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-19 (round 193 — recovery-batch verification audit) **Discovered-By:** AI subagent **Fixed:** 2026-05-19 (round 200 — per-package test-assertion repair) **Evidence:** Round-193 audit of 10 recovery-batch-landed packages (recovery commits b7f8672 + 5c94696) found 6 PASS / **4 FAIL:** - internal/llm (round 161): test-assertion

drift — tests still expected pre-i18n English literal “api_key”, production emits message-ID
internal_llm_wizard_anthropic_apikey_required - internal/logo
(round 163): same drift — tests expected “failed to open” / “failed to decode”, production emits internal_logo_open_source_failed / internal_logo_decode_source_failed - internal/notification (round 167): same drift — tests expected Title literals “Task Completed”, “Task Failed”, “Workflow Completed”, “Workflow Failed”, “Worker Disconnected”, “System Error”, “System Started”; production emits internal_notification_title_* IDs - internal/performance
(round 168): build break — translator.go stdctx.Context vs plain context — fixed inline by parent agent **Root cause:** Recovery-batch commits captured stalled-agent file content but did NOT re-run consuming-test updates + did NOT verify build/test green per-package. **Resolution:** Round-200 subagent updated test assertions in all 3 drifted packages to expect message-ID echoes (internal_<pkg>* prefix). Per-package PASS confirmed (llm: 51.8s, logo: 0.07s, notification: 0.89s, performance: 8.4s). Per CONST-035 mutation-verified one assertion per package: revert to literal → FAIL (production emits the ID), restore → PASS. **Audit reference:** docs/audits/2026-05-19-recovery-batch-verification.md (commit 1badef1).

HXC-003 (ex-ISSUE-007) — CONST-046 i18n migration backlog — CLOSED (migrated to docs/Fixed.md)

Status: Implemented (→ Fixed.md) — see docs/Fixed.md for the full closure record. **Type:** Feature The genuine user-facing (C) string-literal surface is exhausted across all 7 CONST-046 scope areas — helix_code internal/ + cmd/ + applications/ (exhausted rounds 461/462), LLMsVerifier (round 452), helix_qa, and every owned vasic-digital/* + HelixDevelopment/* submodule (rounds 413/441). Hundreds of rounds (~91-462) migrated tens of thousands of literals through i18n seams with paired-mutation anti-bluff tests. The remaining ~55k audit-baseline hits are all OUT of CONST-046 scope (LLM prompt templates, wrapped-error tech strings, identifier tokens, struct-tag keys, format-spec tokens, test fixtures) — documented in docs/audits/2026-05-20-internal-const046-classification.md. Closed by round 463. Section retained as a migration tombstone per §11.4.19 — the authoritative closure narrative is in docs/Fixed.md.

HXQ-001 (ex-ISSUE-008) — helix_qa intermittent TestPerformance flake (host-load-sensitive)

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-19 (round 82) **Discovered-By:** AI subagent **Fixed:** 2026-05-20 (round 325) **Evidence:** helix_qa TestPerformance (three perf tests in pkg/vision/ — TestPerformance_DHash64_Under5msPer1080pFrame, TestPerformance_PHash_Under25msPer1080pFrame, TestPerformance_SSIM_Under5msPer480pFrame) fails intermittently under high host load (concurrent containers + builds). Not a code bug per se; a sensitivity issue — the hard per-frame timing ceilings (5 ms / 25 ms / 5 ms) are only meaningful on a quiescent host. **Resolution:** Decision — **path (b)** (env-var gating) chosen over path (a) (loosen tolerance). Rationale: loosening the timing tolerance would weaken the test's anti-bluff value — a genuine perf regression could then pass. Path (b) preserves the strict assertions while making the flake deterministic: the three tests now check `os.Getenv("HOST_LOAD_DEDICATED")` and `t.Skip("SKIP-OK: #HXQ-001 ...")` honestly when unset, running strict only on a quiescent dedicated host (`HOST_LOAD_DEDICATED=1`). This is the CONST-035-compliant choice. Landed in helix_qa submodule commit 649e2dd + meta .gitmodules pointer bump. docs/test-coverage.md §6.1 documents the env var. Post-fix evidence: `go build ./pkg/vision/... exit 0`, `go vet clean`; `go test -count=1 -run TestPerformance ./pkg/vision/... (unset) → all 3 --- SKIP with SKIP-OK: #HXQ-001 marker`; `HOST_LOAD_DEDICATED=1 go test -count=1 -run TestPerformance ./pkg/vision/... → all 3 --- PASS strict (DHash64 average 741ns, PHash average 88.969µs — well under the 5 ms / 25 ms ceilings).`

HXC-005 — cmd/performance_optimization_standalone/main.go is a CONST-035 simulation bluff

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-20 (round 317 — cmd i18n migration subagent) **Discovered-By:** AI subagent — refused to localize the file because doing so would polish a bluff **Fixed:** 2026-05-20 (round 318) **Evidence:** helix_code/cmd/performance_optimization_standalone/main.go was a package main that printed “ Starting HelixCode Production Performance Optimization” then *simulated* every optimization phase: `// Simulate production optimization phases, time.Sleep(500 * time.Millisecond) per phase, and improvement := 5.0 + rand.Float64()*20.0` — fabricated improvement percentages from a random number generator. No real profiling, no real optimization, no real measurement. The canonical BLUFF-001-

class anti-pattern (CLAUDE.md §3.3 / §6 ANTI-PATTERN 1) — a binary that reports success for work it never performed. Violated CONST-035 / Article XI §11.9. **Resolution:** Decision — **DELETE** (resolution path b). The standalone tool was genuinely obsolete: fully superseded by cmd/performance_optimization/ (snake_case post-CONST-052), which calls the REAL dev.helix.code/internal/performance.PerformanceOptimizer — real runtime.ReadMemStats, real GOMAXPROCS tuning, real before/after measurement — and carries CONST-046 i18n + a unit-test file. git rm -r cmd/performance_optimization_standalone/ removed the dead bluff; stale references purged from docs/COMPREHENSIVE_AUDIT_REPORT.md. Reproduce-before-fix Challenge added at cmd/performance_optimization/bluff_regression_test.go: TestHXC005_BluffStandaloneDirectoryDeleted asserts the obsolete path is gone + the real command survives; TestHXC005_RealOptimizerMeasuresActualMemory allocates a retained 32 MiB buffer and asserts the optimizer's baseline MemoryUsage tracks a genuine runtime.HeapAlloc reading (not an RNG single-digit). Post-fix evidence: go build ./cmd/... exit 0; go test -count=1 -run TestHXC005 ./cmd/performance_optimization/ → both PASS (literal log: optimizer baseline MemoryUsage=33812624 bytes, runtime.HeapAlloc=33802008 bytes — both real measurements, same order of magnitude. No RNG-fabricated improvement percentages.); anti-bluff smoke on the deleted path returns N/A (directory gone).

PAN-001 — panoptic appendString truncates multi-byte UTF-8 runes to bytes (TestResult.MarshalJSON corrupts non-ASCII)

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-19 (round 298 — panoptic enrichment subagent) **Discovered-By:** AI subagent runner-detector against real executor.TestResult.MarshalJSON **Fixed:** 2026-05-19 (round 302 — panoptic submodule commit 24aa627 + meta pointer bump) **Evidence:** panoptic/internal/executor/executor.go:120 — buf = append(buf, byte(r)) in the else branch of appendString casts a rune to a single byte. Multi-byte UTF-8 codepoints (German umlauts, Spanish accents, Japanese CJK, Serbian Cyrillic, Chinese Han) get truncated to one byte each, producing corrupted JSON output. Honestly tracked via the round-298 Challenge runner's executor-marshal:utf8-detector:regression-present PASS line + KNOWN-ISSUE entry in panoptic/docs/test-coverage.md §7. Affects every consumer that JSON-marshals a TestResult containing non-ASCII text. **Resolution:** Replaced buf = append(buf, byte(r)) with buf = utf8.AppendRune(buf, r) (Go 1.21+) + added unicode/utf8 import. Single-line functional fix. Post-fix evidence: go test -race

-count=1 ./internal/executor/... → ok 4.470s; bash challenges/
panoptic_describe_challenge.sh → 39/39 PASS, 0 FAIL; runner
UTF-8 detector flipped from regression-present → fixed (literal log:
PASS [executor-marshal:utf8-detector:fixed] + KNOWN-ISSUE-
RESOLVED: executor.appendJSONString now UTF-8 clean). Closed in
this round.

HXV-002 — LLMsVerifier verification/ package 10 pre-existing test failures

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-20
(round 345 — LLMsVerifier i18n round-12 subagent) **Discovered-**
By: AI subagent — git stash test confirmed the 10 failures
reproduce at submodule HEAD 582ae9c7 (round-336) *without* the
round-345 i18n change, proving pre-existing and unrelated
Resolved: 2026-05-20 (round 348) **Evidence:** go test ./
verification/... in dependencies/HelixDevelopment/LLMsVerifier/llm-
verifier/ reported 10 failures; after round-348 fix ok
digital.vasic.llmsverifier/verification (1.635s), 0 failures, go
build ./... clean. **Resolution:** All 10 failures classified **(A) test-**
assertion drift — every failing test asserted pre-honesty
fabricated behaviour that round-17 commit a6328629 correctly
removed. **No production code changed.** Per-failure
classification table:

#	Test	File
1	TestVerifier_Verify_Success	verification
2	TestVerifier_Verify_ResultScores	verification
3	TestVerifier_Verify_LatencyMetrics	verification
4	TestVerifier_Verify_CodeLanguageSupport	verification
5	TestVerifier_Verify_CodeCapabilities	verification
6	TestVerifier_Verify_ModelStatusFlags	verification
7	TestVerifier_Verify_ContextCancellation	verification

#	Test	File
8	TestVerifier_Verify_MultipleRequests	verification
9	TestCodeVerificationService_TestCodeVisibility_Error	code_veri
10	TestCodeVerificationService_VerifyModelCodeVisibility_ServerError	code_veri

Mirrors HXV-001 round-323's classification approach. The production code (verification.go round-17 ErrVerificationNotWired, code_verification.go error propagation + zero-response→failed) was already honest; only the stale test assertions needed re-keying to certify it.

HXC-006 — HelixCode Speed Programme — CLOSED (migrated to docs/Fixed.md)

Status: Implemented (→ Fixed.md) — see docs/Fixed.md for the full closure record. **Type:** Feature All 6 phases / 31 tasks landed; CONST-048 coverage ledger at docs/research/speed/05-coverage-ledger.md (29 PASS + 2 PARTIAL + 0 DEFERRED). Closed by P5-T04 round 400. Section retained as a migration tombstone per §11.4.19 — the authoritative closure narrative is in docs/Fixed.md.

HXC-007 — Constitution §11.4.68/70-74 cascade + meta-pointer bump — CLOSED (migrated to docs/Fixed.md)

Status: Completed (→ Fixed.md) — see docs/Fixed.md for the full closure record. **Type:** Task Cascade verified complete (round 403): constitution 584b3ee→34a82b3, all 6 rules cascaded to the meta-repo + 67 owned submodules, meta pointer confirmed at 34a82b3. Section retained as a migration tombstone per §11.4.19.

HXC-008 — CONST-055 G1 governance gaps surfaced by post-constitution-pull validation sweep — CLOSED (migrated to docs/Fixed.md)

Status: Fixed (→ Fixed.md) — see docs/Fixed.md for the full closure record. **Type:** Bug Both gaps fixed (round 403): (a) submodule_owned.txt corrected HelixDevelopment/Models→models; (b) CONST-047..057 cascaded into helix_qa/CONSTITUTION.md, §11.4.69 cascaded into VisionEngine/CONSTITUTION.md. verify-governance-cascade.sh → 0 failures; verify-all-constitution-rules.sh → 6 gates / 0 failures. Section retained as a migration tombstone per §11.4.19.

HXC-009 — Owned-submodule GitHub ↔ GitLab mirror-divergence reconciliation — CLOSED (migrated to docs/Fixed.md)

Status: Completed (→ Fixed.md) — see docs/Fixed.md for the full closure record. **Type:** Task Reconciliation verified complete (round 403): helix_qa, VisionEngine, LLMProvider, challenges, containers, DocProcessor all reconciled via merge-first (CONST-061 / §11.4.71), all owned submodules converged + pushed to all upstreams. Section retained as a migration tombstone per §11.4.19.

HXC-010 — End-to-end Kimi CLI + Qwen Code CodeGraph verification — CLOSED (migrated to docs/Fixed.md)

Status: Completed (→ Fixed.md) — see docs/Fixed.md for the full closure record. **Type:** Task Operator supplied OpenAI-compatible router credentials (2026-05-21). Both cg-challenge-05-kimi.sh and cg-challenge-07-qwen.sh re-run produce **true tier-1 PASS** — each agent genuinely invoked codegraph_search and received real graph data from the scanned HelixCode code-graph. Kimi driven via an openai_legacy provider, Qwen via --auth-type openai, both against SiliconFlow; API keys injected via environment variables only, never written to any tracked file (CONST-042). Section retained as a migration tombstone per §11.4.19.

HXC-013 — Adopt SQLite-backed single-source-of-truth for workable items (§11.4.93/95)

Status: Queued **Type:** Feature **Discovered:** 2026-05-28 (constitution pull 7f738df→15cd4bc) **Discovered-By:** AI (post-pull awareness review) **Scope:** §11.4.93 + §11.4.95 require a tracked docs/workable_items.db as authoritative, with a Go binary doing bidirectional md↔db sync. Critical findings: the constitution's constitution/scripts/workable-items/ binary is a NON-FUNCTIONAL Phase-2 scaffold (every subcommand prints "not yet implemented") — adoption requires building the parser/renderer UPSTREAM per §11.4.74 (triggers §11.4.26); .gitignore:162 blanket *.db must gain !docs/workable_items.db; HelixCode lacks generate_issues_summary.sh/sync_issues_docs.sh. Effort ~5-9 subagent-days. **Operator decisions:** id strategy (recommend keep HXC-NNN in free-text atm_id col); .gitignore negation; upstream-first vs wait; round-trip tolerance.

HXC-014 — Stress + chaos test coverage (§11.4.85)

Status: In progress **Type:** Task **Discovered:** 2026-05-28 (constitution pull) **Discovered-By:** AI **Scope:** §11.4.85 requires every fix/improvement to ship stress (sustained-load $N \geq 100/\geq 30s$ p50/p95/p99 + concurrency $N \geq 10$ + boundary) + chaos (process-death/network-fault/input-corruption/resource-exhaustion/state-corruption) suites with captured evidence. **Batch 1 DONE (2026-05-28, commits 76586014 + a9f883c6):** Go-native helper helix_code/tests/stresschaos/ (RunSustainedLoad p50/p95/p99→latency.json; RunConcurrent ≥ 10 goroutines + deadlock/leak guards; ChaosKill/CorruptInput/ResourcePressure injectors $\leq 128MB$ §12.6-safe; evidence→qa-results/ gitignored CONST-053) + 7 paired §1.1 meta-tests proving the harness can't bluff (deadlock/leak/error-rate/below-floor/panic all DETECTED). First 2 targets done: internal/worker + internal/llm/load_balancer under -race. **The chaos tests SURFACED + FIXED 2 REAL production bugs:** (1) WorkerPool.AssignTask RWMutex-reentrancy deadlock (double-RLock), (2) GetPoolStats data race on PoolWorker.Status. make stress-chaos + make stress-chaos-meta added. Conductor independently re-ran meta-tests → PASS. **Batch 2 DONE (2026-05-28, commit 0505c337):** internal/task (TaskManager/TaskQueue — sustained p50=0.21ms, concurrent 16×120 gDelta=0, state-corruption + input-corruption chaos) + internal/session (TranscriptStore/ResumeFinder, real disk I/O — sustained, concurrent 12×60 no-loss, input-corruption + process-death-mid-append crash-consistent recovery). 8 tests PASS under

-race, captured evidence. No new bugs (components robust). All 4 in-process targets (worker/load_balancer/task/session) now covered. **Batch 3 DONE (2026-05-28, subagent-driven §11.4.70):** internal/event + internal/memory + internal/context (3 parallel disjoint-scope subagents). **2 MORE REAL production bugs SURFACED + FIXED:** (1) internal/event/bus.go — a panicking event handler crashed the whole process (in async dispatch the handler runs in its own goroutine; an unrecovered panic took down every goroutine). Fix: EventBus.invokeHandler recover-wrapper routed through all 3 dispatch sites (Publish async/sync + PublishAndWait) → graceful degradation. (2) internal/memory/manager.go — GetConversation/GetActive/GetAll/GetBySession/GetRecent/Search returned the LIVE stored *Conversation pointer out from under the RLock; callers read conv.Messages/MessageCount concurrently with AddMessage/Clear writers → genuine DATA RACE the map-only RWMutex didn't cover. Fix: return conv.Clone() snapshots + repaired Conversation.Clone() which silently dropped CharacterID/UserID/CharMessages/Version/Status. internal/context: no bug (RWMutex discipline held), full coverage added. All consumers of the memory read methods confirmed read-only (context builder / persistence snapshot / listing) → no regression. Each fix proven anti-bluff by reverting → test FAILs → restore → PASS. make stress-chaos target extended to include task/session/event/memory/context; full target PASSES under -race. 7 in-process targets now covered. **Batch 4 DONE (2026-05-28, subagent-driven §11.4.70):** internal/rules + internal/repomap + internal/discovery (3 parallel disjoint-scope subagents). No new production bugs — all three components already correctly RWMutex-guarded and survived concurrent churn + Clear/cancel races + malformed/corrupt input + bounded memory pressure with no panic/race/deadlock/leak. Each suite proven genuinely fail-capable (anti-bluff): rules → strip AddProjectRule lock → DATA RACE FAIL; repomap → write results[0] in parseFilesParallel → DATA RACE FAIL; discovery → strip port-range check → boundary FAIL; all restored byte-identical. discovery deliberately targets in-process registry/config/JSON-decode paths (not flaky UDP-multicast/net.Listen). 10 in-process targets now covered; make stress-chaos extended to all 10, full target PASSES under -race. **Batch 5 DONE (2026-05-28, subagent-driven §11.4.70):** internal/tools + internal/workflow + internal/hooks (3 parallel disjoint-scope subagents). **3 MORE REAL production bugs SURFACED + FIXED, all in internal/hooks:** (1) executor.go executeSync/executeAsync called hook.Execute with no recover() — a panicking async hook handler crashes the whole process (same bug class as the event-bus fix); fixed via Executor.runHandler recover-wrapper → graceful degradation. (2) manager.go TriggerEvent/TriggerEventAndWait/TriggerEventSync held m.mu.RLock() then called GetByType() which re-locks the non-reentrant RWMutex — a writer (Register/Unregister) queuing between the two RLock acquisitions deadlocks both (same bug class as the worker batch-1 deadlock); fixed by dropping the redundant outer RLock (GetByType already

returns a defensive copy under its own lock — verified). (3) Register(nil) nil-pointer panic (Validate dereferences nil receiver); fixed with a nil guard + CONST-046-compliant i18n key internal_hooks_nil. internal/tools (ToolRegistry + real fs/shell tools — command-injection + CONST-033 power-mgmt commands confirmed REFUSED by the security gate before os/exec) and internal/workflow (state-machine + executor + BackgroundManager) had no bugs, full coverage added. Each fix anti-bluff-proven: revert → FAIL (panic / 30s deadlock timeout / FATAL) → restore → PASS. 13 in-process targets now covered; make stress-chaos extended to all 13, full target PASSES under -race. **Batch 6 DONE (2026-05-28, subagent-driven §11.4.70):** internal/agent + internal/monitoring + internal/commands (3 parallel disjoint-scope subagents). **3 MORE REAL production bugs SURFACED + FIXED:** (1) internal/agent/agent.go — AgentRegistry.agents map had NO mutex; Register/Unregister write it while Get/List/Count/GetByType/GetByCapability read it, and the Coordinator delegates without holding c.mu → unsynchronised concurrent map access = guaranteed fatal error: concurrent map read and map write under load. Fix: added sync.RWMutex (writers Lock, readers RLock, never reentrant). (2) internal/monitoring/monitor.go:48 — CollectMetrics called collector.Collect() under the write lock with no recover(); a panicking collector (HTTP scrape / /proc read fault) crashes the process. Fix: safeCollect recover-wrapper + i18n key internal_monitoring_collector_panic. (3) internal/commands/executor.go — Executor.Execute called cmd.Execute() with no recover(); a panicking slash/markdown/third-party command crashes the host CLI/server. Fix: executeRecovered wrapper + i18n key internal_commands_command_panicked. All CONST-046-compliant (no hardcoded English). Each fix anti-bluff-proven: revert → DATA RACE / concurrent-map-crash / panic → restore → PASS. **16 in-process targets now covered; 8 real production bugs fixed this session** (worker/load_balancer batch 1; event/memory batch 3; hooks×3 batch 5; agent/monitoring/commands batch 6); make stress-chaos extended to all 16, full target PASSES under -race. **Batch 7 DONE (2026-05-28, subagent-driven §11.4.70):** internal/mcp + internal/notification + internal/performance (3 parallel disjoint-scope subagents). **4 MORE REAL production bugs SURFACED + FIXED:** (1+2) internal/mcp/server.go handleCallTool invoked tool.Handler with no recover() (handleSession dispatches each message via go handleMessage, so a panicking tool handler crashes the process) + internal/mcp/lifecycle.go Client.setState called the caller-supplied onEvent callback inline with no recover (panic crashes the Connect/Close/recvLoop goroutine); fixed via invokeToolHandler recover-wrapper + i18n key internal_mcp_server_tool_handler_panicked + callback recover. (3) internal/notification/engine.go sendToChannels called channel.Send with no recover — a panicking channel crashes the process when dispatched from a NotificationQueue worker goroutine; fixed via sendOne recover-wrapper. (4) internal/performance/optimizer.go declared po.mutex but NEVER used it —

StartProductionOptimization wrote po.optimizations while getOptimizationsByType iterated it unsynchronised → fatal error: concurrent map iteration and map write; fixed with Lock on write + RLock on read. Each anti-bluff-proven: revert → panic/FATAL/DATA RACE → restore → PASS. **19 in-process targets now covered; 12 real production bugs fixed this session** (+mcp×2, notification, performance in batch 7); make stress-chaos extended to all 19, full target PASSES under -race. **Batch 8 DONE (2026-05-28, subagent-driven \$11.4.70):** internal/security + internal/providers + internal/llm-manager (3 parallel disjoint-scope subagents). **6 MORE REAL production bugs SURFACED + FIXED:** (1+2) internal/security/translator.go — package-level translator var read by tr() + written by SetTranslator() with NO mutex (data race; scans run in background goroutines) + tr() called translator.T() with no recover (panicking injected translator crashes the scan-worker goroutine); fixed with RWMutex + recover. (3+4) internal/providers/ai_integration.go — Initialize held ai.mu.Lock() then called NewConversationManager→GetProvider→ai.mu.RLock() = non-reentrant self-deadlock hanging the process (fixed: release write lock before building managers, re-acquire to store) + vector_integration.go NewVectorIntegration(nil) left config nil → Initialize SIGSEGV (fixed: nil-config default mirroring NewMemoryIntegration; codifying-the-crash test assertion corrected). (5) internal/llm/model_manager.go:341 — calculateHardwareCompatibility called hardwareDetector.Detect() (mutates detector state) under only RLock → data race across concurrent SelectOptimalModel callers; fixed with sync.Once (host hardware static). (6) **load_balancer follow-up (conductor-fixed, surfaced by the llm subagent):** internal/llm/load_balancer.go Stop() never cancelled the collectStats goroutine (leak when Start got context.Background()) + collectPerformanceStats deref'd lb.manager with no nil-guard → nil-pointer panic after the 10s ticker; fixed with a cancelStats CancelFunc cancelled in Stop() + nil-guard. Also resolved the file's pre-existing CWE-338 (math/rand→crypto/rand uniform provider selection, dropping the deprecated rand.Seed) flagged by the semgrep gate. Full internal/llm package now PASSES under -race with NO skip (previously the nil-manager test had to be skipped). Each fix anti-bluff-proven: revert → DATA RACE / 15s-deadlock-timeout / SIGSEGV / FATAL → restore → PASS. **22 in-process targets now covered; 18 real production bugs fixed this session. Batch 9 DONE (2026-05-28, subagent-driven \$11.4.70):** internal/editor + internal/template + internal/cognee (3 parallel disjoint-scope subagents). **2 MORE REAL production bugs SURFACED + FIXED in internal/template/manager.go:** Register/Update/Delete invoked user OnCreate/OnUpdate/OnDelete callbacks (a) with no recover() → a panicking callback crashes the process (esp. when Register runs on a goroutine), and (b) the On* registrars appended to the callback slices with NO lock while the trigger paths iterated them under lock → data race. Fix: snapshotCallbacks under lock + fireCallbacks panic-isolated

AFTER unlock (also prevents re-entrant-callback deadlock); On* registrars now lock-guarded. internal/editor (CodeEditor/WholeEditor/SearchReplaceEditor/LineEditor/DiffEditor over real t.TempDir files) and internal/cognee (ServiceCache/ServiceStatistics/event-handler fan-out in-process; network Client paths honestly out of scope) had no bugs, full coverage added. Each anti-bluff-proven: revert → panic / DATA RACE → restore → PASS. **25 in-process targets now covered; 20 real production bugs fixed this session. Batch 10 DONE (2026-05-28, subagent-driven §11.4.70):** internal/focus + internal/config + internal/persistence (3 parallel disjoint-scope subagents). **5 MORE REAL production bugs SURFACED + FIXED:** (1+2) internal/focus/manager.go — CreateChain/SetActiveChain/DeleteChain invoked onCreate/onActivate/onDelete callbacks in a loop with no recover() WHILE holding m.mu.Lock() → panicking callback crashes the process + a callback re-entering the Manager deadlocks the non-reentrant RWMutex; fixed via invokeCallbacks (recover) + snapshot-then-release-lock-before-dispatch. (3) internal/config/config.go — ConfigManager had NO mutex at all; GetConfig/loadConfig/ImportConfig/UpdateConfig/ResetToDefaults/saveConfig read+wrote m.config unguarded while the ConfigAPI HTTP handlers drive GET /config concurrently with POST /config/reload → data race; fixed with RWMutex + copy-on-write UpdateConfig + load/save split into public(locking)+Locked(caller-holds) helpers (non-reentrant-safe) + decode-into-fresh-struct-swap-on-success. (4+5) internal/persistence/store.go — SaveAll/LoadAll/triggerError invoked user callbacks with no recover (process crash) + On* registrars appended to callback slices with no lock while Save/Load iterated under lock (data race); fixed with invoke*Callback recover-wrappers + lock-guarded registration + snapshot-dispatch-outside-lock + 3 i18n keys. (Note: a transient build-break appeared mid-batch when config's in-flight edit referenced its not-yet-added Locked helpers; the persistence subagent correctly verified in an isolated git worktree per §11.4.84/§11.4.96; combined final state builds + passes -race, conductor re-verified.) internal/focus, config, persistence all CONST-046-compliant. Each anti-bluff-proven: revert → panic / deadlock / DATA RACE → restore → PASS. **28 in-process targets now covered; 25 real production bugs fixed this session. Batch 11 DONE (2026-05-28, subagent-driven §11.4.70):** internal/auth + internal/deployment + internal/project. **4 MORE REAL production bugs SURFACED + FIXED:** (1) SECURITY internal/auth/auth.go VerifyJWT — unchecked type assertions claims["username"].(string)/claims["email"].(string); a validly-signed token carrying a missing/numeric/null username/email claim PANICS the process (crash-on-untrusted-input DoS — a single forged request). Fixed with comma-ok assertions → ErrTokenInvalid. (2+3) internal/deployment/production_deployer.go addNotification appended to shared status with a declared-but-UNUSED mutex (data race) + deployment/translator.go package-var race; fixed with the mutex + translatorMu. (4) internal/

project/manager.go GetActiveProject wrote m.activeProject under only RLock (lazy-scan write under read-lock) → data race; fixed with exclusive Lock + re-check. Each anti-bluff-proven: revert → panic / DATA RACE → restore → PASS. **31 in-process targets now covered; 29 real production bugs fixed this session.**

HXC-014b — Systemic unguarded i18n translator.go data-race + panic-crash (cross-package)

Status: Fixed (→ Fixed.md) **Type:** Bug **Closure (2026-05-28, subagent-driven §11.4.70):** all 52 remaining unguarded internal/*/translator.go files fixed (3 parallel disjoint-group subagents, 18+17+17) to match the proven internal/security + internal/deployment guarding pattern — added translatorMu sync.RWMutex (Lock in SetTranslator, RLock-snapshot in tr) + a recover() in tr (named return) degrading a panicking translator to the message ID, and removed the inline if translator==nil { translator = Noop } write-on-read-path. 54/54 translator.go files now guarded; whole internal/...+cmd/... tree builds clean; gofmt clean. Anti-bluff proof: new internal/logging/translator_race_test.go hammers SetTranslator concurrently with tr (16×300) + a panicking translator; §1.1 paired mutation (strip the guard) → WARNING: DATA RACE + panic + FAIL → restore → PASS. Behaviour-preserving (only thread-safety + panic-isolation; same msgID fallback + nil→Noop reset). **Discovered:** 2026-05-28 (HXC-014 batch 8+11 — same defect found independently in internal/security AND internal/deployment translator.go) **Severity:** Medium (latent: requires SetTranslator() concurrent with tr()); existing -race tests pass because SetTranslator is boot-only in practice) **Scope:** ~50 packages under helix_code/internal/*/translator.go share a copy-pasted CONST-046 resolver seam with (a) an UNGUARDED package-level var translator <pkg>i18n.Translator — SetTranslator writes it + tr() reads it (and self-heals translator = Noop{} on the read path, a write) with NO mutex → data race if SetTranslator is ever called concurrently with tr(); (b) NO recover() around translator.T() → a panicking injected/buggy Translator crashes the emitting goroutine (process-wide). The FIX PATTERN is already proven in internal/security/translator.go + internal/deployment/translator.go: add translatorMu sync.RWMutex (Lock in SetTranslator, RLock-snapshot in tr) + a recover() in tr degrading to the message ID (named return). Mechanical, identical per file; ~50 files remain (event/memory/context/rules/repomap/discovery/tools/workflow/hooks/agent/monitoring/commands/mcp/notification/performance/providers/llm/editor/template/cognee/focus/config/persistence/auth/project + ~25 more incl. approval/clarification/planner/plantree/plugins/quality/redis/render/roocode/secrets/session/verifier/voice/worker/workspace/etc.). NOTE: several of those

packages' OTHER concurrency bugs were already fixed in batches 3-11, but most still carry the unguarded translator seam. **Best fixed as a single carefully-verified mechanical sweep (build + -race after) rather than rushed — tracked separately so it is not lost. Remaining (infra batch + long-tail):** internal/redis + internal/database + internal/persistence (real PG/Redis via make test-infra-up), internal/server (HTTP/WS DDoS-class load), internal/llm provider endpoints — all need the docker-compose stack up; then the long-tail ~40-package sweep. Effort remaining multi-week incremental. **HXC-014a** (empty TestProviderStress stub) already FIXED (f464adb0). **Operator decision deferred:** promoting tests/stresschaos/ into the constitution submodule for cross-project reuse (triggers §11.4.26 cross-project workflow) — interim home is project-local.

HXC-015 — Cross-platform parity (§11.4.81)

Status: Queued **Type:** Task **Discovered:** 2026-05-28 (constitution pull) **Discovered-By:** AI **Scope:** §11.4.81 requires per-OS equivalents (runtime dispatch) for platform-specific primitives + honest kernel-gap citations. Gaps: shell scripts rarely uname-dispatch, no CM-CROSS-PLATFORM-PARITY gate, no supported-platforms manifest, shell/sandbox.go rlimits a never-applied stub. Sub-defect **HXC-015a (7 platform Assert(true, "...skipped") fake-skips) already FIXED in commit f464adb0** (honest v.Skip on runtime.GOOS). This item tracks the remaining parity gate + manifest + rlimit work. Open decision: which non-Linux platforms have test hardware (else honest OPERATOR-BLOCKED). **Doc-fix:** root CLAUDE.md §3.2.1 prose says inner module is helix_code/ helix_code/; actual is helix_code/ — correct the prose.

HXC-016 — §11.4.69-97 governance cascade into owned submodules (CONST-047/§3) — CLOSED (→ Fixed.md)

Status: Completed (→ Fixed.md) — see docs/Fixed.md for the full closure record. **Type:** Task **Discovered:** 2026-05-28 (constitution pull) **Discovered-By:** AI **Closure (2026-05-28):** all 24 anchors (§11.4.69 + §11.4.75-97) cascaded into the 5 root govfiles (27929ae1) AND all ~68 owned-submodule CONSTITUTION/ CLAUDE/AGENTS/QWEN files (batches 1-7: ef4b3986/a864039d/ e4046668/3adb2e63/464b2401/b4ad4f50/053fd731). A loose-grep false-match (the §11.4.93 body cites §11.4.95) had skipped the

§11.4.95 heading in batch-1-6 submodules — repaired by fix-up A/B/C (79478ed5/903b9225/a9a1a6a1) + the gate tightened to the ## §11.4.NN – heading marker (d2165bf7). 4 HelixDevelopment submodules regressed to the wrong branch (main vs canonical master) by the cascade reset — repaired (b4b790ea). Gate submodule-scope enabled (9031368d). Final verify-governance-cascade.sh → **0 failures**, 204 submodule covenant-114 PASS lines; paired §1.1 mutation proven (strip §11.4.95 → 1 failure → restore → 0). Section retained as a migration tombstone per §11.4.19.

HXC-017 — CodeGraph own-org submodule indexing + update automation (§11.4.79/80) — CLOSED (→ Fixed.md)

Status: Completed (→ Fixed.md) — see docs/Fixed.md for the full closure record. **Type:** Task **Discovered:** 2026-05-28 (constitution pull) **Discovered-By:** AI **Closure (2026-05-28, commits 176fe07b + 551552f7 + 876b3b36):** .codegraph/config.json blanket dependencies/** exclude replaced with 3 specific third-party excludes (LLama_CPP/Ollama/HuggingFace_Hub) so own-org dependencies/vasic-digital/** + dependencies/HelixDevelopment/** are now INCLUDED; credential excludes **(/.env,.key,.pem,secrets) added. Root .gitignore fixed so .codegraph/config.json is TRACKED (§11.4.78 — it had been blanket-ignored). Re-index (codegraph index . exit 0): Files 39,024→76,044, Nodes 624,103→1,255,974, Edges 1.64M→3.96M.** §11.4.79 anti-bluff probe (independently re-verified by conductor):** codegraph query EventBus → dependencies/vasic-digital/event_bus/pkg/bus/bus.go:85; helix_memory → dependencies/HelixDevelopment/helix_memory/...; third-party llama filtered to LLama_CPP → empty. docs/codegraph/Status.md + Status_Summary.md created (§11.4.80; weekly automation inherited by reference from constitution codegraph_update.sh/codegraph_sync.sh). Section retained as a migration tombstone per §11.4.19.

HXC-018 — Obsolete status (§11.4.90) + summary-doc clarity (§11.4.91) tracker tooling

Status: Queued **Type:** Task **Discovered:** 2026-05-28 (constitution pull) **Discovered-By:** AI **Scope:** §11.4.90 adds terminal Obsolete (→ Fixed.md) status + Obsolete-Details line + colorizer cell-status-

obsolete; §11.4.91 forbids anti-pattern summary one-liners (“Composes with” etc.) and requires generators to refuse them. Extend HelixCode’s tracker + (currently-absent) summary generators + colorizer. Couples with HXC-013 (DB schema includes Obsolete).

HXC-019 — docs/qa/ end-user evidence tree (§11.4.83)

Status: Queued **Type:** Task **Discovered:** 2026-05-28 (constitution pull) **Discovered-By:** AI **Scope:** §11.4.83 requires every shipped feature to carry a recorded e2e transcript + materials under docs/qa/<run-id>; release gate refuses feature commits lacking it. Establish the tree + gate.

HXC-022 — test_bank platform + integration packages do not compile (pre-existing) — CLOSED (→ Fixed.md)

Status: Fixed (→ Fixed.md) — see docs/Fixed.md for the full closure record. **Type:** Bug **Discovered:** 2026-05-28 (anti-bluff sweep during HXC-021 fix) **Discovered-By:** AI (captured: go build ./platform/... ./integration/... in helix_code/tests/e2e/test_bank) **Closure (2026-05-28, commit 02b3081c):** all ~11 named declared and not used half-written stubs COMPLETED with real assertions (created-resource IDs → assert non-empty; metric values → assert non-nil; 2 vestigial unsent request bodies removed); root-dir package collision resolved by git mv performance_security_tests.go → performance/ subpackage; unmasked pre-existing core/ defects (duplicate GetCoreTests, unused imports) fixed too. go build ./... exit 0 (whole module), go vet ./... clean — independently re-verified. HXC-021 runtime-verified through the now-compiling banks: platform SKIP=3 (honest “not running on macOS/Windows/ARM”), integration SKIP=2 (honest “Ollama not reachable”/“OPENAI_API_KEY not configured”), no fake PASS/green-empty. Section retained as a migration tombstone per §11.4.19.

HXC-023 — Assert(true,...) / AssertTrue(true,...) literal-true bluffs across test_bank — CLOSED (→ Fixed.md)

Status: Fixed (→ Fixed.md) — see docs/Fixed.md for the full closure record. **Type:** Bug **Discovered:** 2026-05-28 (surfaced while fixing HXC-022) **Discovered-By:** AI **Closure (2026-05-28, commits 8e80e0c0 + b514f8bb):** ALL literal-true PASS-bluffs across the e2e test banks replaced with real assertions or honest skips — batch 1 (core/additional_tests.go, 41 fixed) + batch 2 (distributed 12, integration 11, platform 5, core/tests.go 4, performance 1 = 33). Pattern: mislabelled “X succeeded” branches that fired on non-2xx → assert the expected 2xx; 401/403/429 branches → assert that exact status; feature-404 branches → honest v.Skip(reason) (§11.4.3); the 4 legitimate “Running on ” positive-platform asserts left untouched. Verification: go build ./... + go vet ./... exit 0; full-tree grep for literal-true bluffs = 0; runtime harness (down server) → all changed cases HONEST-FAIL, 0 green-empty. Section retained as a migration tombstone per §11.4.19.

Last updated: 2026-05-28 — constitution submodule pulled 7f738df→15cd4bc (§11.4.79-97); HXC-013..019,022 filed (open: SQLite-DB / stress+chaos / cross-platform / submodule-cascade / codegraph-own-org / obsolete+summary-tooling / docs-qa / test bank-noncompile); HXC-021 + HXC-014a + HXC-015a FIXED→Fixed.md (commit f464adb0 — fake-skip Assert(true) bluffs + empty stress stub → honest SKIP); CONST-052/HXC-001 leaf-rename programme COMPLETE (Phases 1-4), Phase 5 org-grouping dirs kept as namespace carve-outs per operator decision 2026-05-28 → HXC-001 closeable. Prior: 2026-05-20 (round 463 — HXC-003 closed Implemented (→ Fixed.md) and migrated to docs/Fixed.md: the CONST-046 i18n migration campaign is concluded — the genuine user-facing (C) string-literal surface is exhausted across all 7 scope areas (helix_code internal/+cmd/+applications/, LLMsVerifier, helix_qa, all owned vasic-digital/+HelixDevelopment /* submodules); ~91-462 rounds migrated tens of thousands of literals with paired-mutation anti-bluff tests; remaining ~55k audit hits are all out of CONST-046 scope per docs/audits/2026-05-20-internal-const046-classification.md. Open set is now HXC-001 (CONST-052 renames — Task, In progress) + HXC-010 (Kimi/Qwen codegraph e2e — Operator-blocked Task)). Previous round 402 — HXC-011 closed Fixed (→ Fixed.md): the helix_qa runner’s run path on the desktop platform now genuinely executes a bank case’s shell: action via os/exec. Round 400 — speed-programme close-out: HXC-006 closed Implemented (→ Fixed.md). To update Issues_Summary.md mechanically, run scripts/*

*generate_issues_summary.sh (TODO: create — currently this
Issues.md is the source of truth and Summary is hand-
maintained).*